

Module #3 – Introduction to OOPS Programming

THEORY EXERCISES ANSWERS

1. Introduction to C++

1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

Procedural Programming (POP)	Object-Oriented Programming (OOP)
Focuses on functions	Focuses on objects
Uses top-down approach	Uses bottom-up approach
Data is less secure	Data is more secure
Difficult for large projects	Easier for large projects
No inheritance support	Supports inheritance

2. List and explain the main advantages of OOP over POP.

1. Better data security through encapsulation.
 2. Code reusability using inheritance.
 3. Easier maintenance and debugging.
 4. Programs become modular and organized.
 5. Supports real-world modeling.
-

3. Explain the steps involved in setting up a C++ development environment.

1. Install a C++ IDE such as Dev C++ or CodeBlocks.
2. Install a compiler like GCC.
3. Create a new C++ project.
4. Write and save the program.
5. Compile the code.

6. Run the program and check output.

4. What are the main input/output operations in C++? Provide examples.

- `cin` is used for input.
- `cout` is used for output.

Example:

```
int age;  
cin >> age;  
cout << age;
```

2. Variables, Data Types, and Operators

1. What are the different data types available in C++? Explain with examples.

Data Type	Example
int	int a = 10;
float	float b = 3.5;
char	char c = 'A';
double	double d = 10.55;
string	string name = "Rahul";

2. Explain the difference between implicit and explicit type conversion in C++.

- Implicit conversion is automatically performed by the compiler.
- Explicit conversion is manually performed using type casting.

Example:

```
float x = 10;  
int y = (int)9.8;
```

3. What are the different types of operators in C++? Provide examples of each.

1. Arithmetic Operators (+, -, *, /)
 2. Relational Operators (>, <, ==)
 3. Logical Operators (&&, ||)
 4. Bitwise Operators (&, |)
-

4. Explain the purpose and use of constants and literals in C++.

- Constants are fixed values that cannot be changed.
- Literals are constant values directly written in code.

Example:

```
const float PI = 3.14;
```

3. Control Flow Statements

1. What are conditional statements in C++? Explain the if-else and switch statements.

Conditional statements control program flow based on conditions.

if-else Example

```
if(a > b)
    cout << "A is greater";
else
    cout << "B is greater";
```

switch Example

```
switch(choice) {
    case 1:
        cout << "One";
        break;
}
```

2. What is the difference between for, while, and do-while loops in C++?

Loop	Description
for	Used when number of iterations is known
while	Condition checked before execution
do-while	Executes at least once

3. How are break and continue statements used in loops? Provide examples.

- `break` terminates the loop.
- `continue` skips the current iteration.

Example:

```
for(int i = 1; i <= 5; i++) {  
    if(i == 3)  
        break;  
}
```

4. Explain nested control structures with an example.

Nested control structures mean using one loop or condition inside another.

Example:

```
for(int i = 1; i <= 3; i++) {  
    for(int j = 1; j <= 3; j++) {  
        cout << "*";  
    }  
}
```

4. Functions and Scope

1. What is a function in C++? Explain declaration, definition, and calling.

A function is a block of code that performs a specific task.

- Function declaration tells the compiler about the function.
 - Function definition contains actual code.
 - Function calling executes the function.
-

2. What is the scope of variables in C++? Differentiate between local and global scope.

Local Variable	Global Variable
Declared inside function	Declared outside function
Accessible only inside function	Accessible throughout program

3. Explain recursion in C++ with an example.

Recursion is a process where a function calls itself.

Example:

```
int factorial(int n) {  
    if(n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

4. What are function prototypes in C++? Why are they used?

Function prototypes declare functions before `main()`.

Example:

```
int add(int, int);
```

They help the compiler understand function return type and parameters.

5. Arrays and Strings

1. What are arrays in C++? Explain single-dimensional and multi-dimensional arrays.

Arrays store multiple values of the same data type.

- Single-dimensional array stores data in one row.
 - Multi-dimensional array stores data in rows and columns.
-

2. Explain string handling in C++ with examples.

Strings are used to store text.

Example:

```
string name = "Rahul";
```

3. How are arrays initialized in C++? Provide examples of 1D and 2D arrays.

1D Array

```
int arr[3] = {1, 2, 3};
```

2D Array

```
int matrix[2][2] = {{1,2},{3,4}};
```

4. Explain string operations and functions in C++.

Common string operations:

- `length()` → Finds string length
- `append()` → Adds strings
- `substr()` → Extracts substring

Example:

```
string s = "Hello";  
cout << s.length();
```

6. Introduction to Object-Oriented Programming

1. Explain the key concepts of Object-Oriented Programming (OOP).

1. Class
 2. Object
 3. Encapsulation
 4. Inheritance
 5. Polymorphism
 6. Abstraction
-

2. What are classes and objects in C++? Provide an example.

- Class is a blueprint.
- Object is an instance of a class.

Example:

```
class Student {  
public:  
    int marks;  
};  
  
Student s1;
```

3. What is inheritance in C++? Explain with an example.

Inheritance allows one class to acquire properties of another class.

Example:

```
class A {  
};  
  
class B : public A {  
};
```

4. What is encapsulation in C++? How is it achieved in classes?

Encapsulation means binding data and functions together inside a class.

It is achieved using private and public access specifiers.

1. Introduction to C++

Experiment 1: Hello World Program

Aim:

To display “Hello, World!” using C++.

Program:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

Conclusion:

The program was executed successfully.

Experiment 2: Basic Input/Output

Aim:

To accept user input and display a personalized greeting.

Program:

```
#include <iostream>
using namespace std;

int main() {
```



```

    string name;
    int age;

    cout << "Enter your name: ";
    cin >> name;

    cout << "Enter your age: ";
    cin >> age;

    cout << "Hello " << name << "! You are " << age << " years old.";

    return 0;
}

```

Conclusion:

The program successfully accepted and displayed user input.

Experiment 3: POP vs OOP Comparison

Aim:

To compare Procedural Programming and Object-Oriented Programming.

POP Program:

```

#include <iostream>
using namespace std;

int main() {
    int length, breadth;

    cout << "Enter length and breadth: ";
    cin >> length >> breadth;

    cout << "Area = " << length * breadth;

    return 0;
}

```

OOP Program:

```

#include <iostream>
using namespace std;

class Rectangle {
    int length, breadth;

public:

```

```
void input() {
    cout << "Enter length and breadth: ";
    cin >> length >> breadth;
}

void area() {
    cout << "Area = " << length * breadth;
}

};

int main() {
    Rectangle r;

    r.input();
    r.area();

    return 0;
}
```

Conclusion:

The difference between POP and OOP was understood successfully.

Experiment 4: Sum of Two Numbers

Aim:

To calculate the sum of two numbers.

Program:

```
#include <iostream>
using namespace std;

int main() {
    int a, b;

    cout << "Enter two numbers: ";
    cin >> a >> b;

    cout << "Sum = " << a + b;

    return 0;
}
```

Conclusion:

The program successfully calculated the sum of two numbers.

2. Variables, Data Types and Operators

Experiment 1: Variables and Constants

Aim:

To demonstrate variables and constants in C++.

Program:

```
#include <iostream>
using namespace std;

int main() {
    int age = 18;
    float marks = 89.5;
    char grade = 'A';
    const float PI = 3.14;

    cout << age << endl;
    cout << marks << endl;
    cout << grade << endl;
    cout << PI;

    return 0;
}
```

Conclusion:

The use of variables and constants was understood successfully.

Experiment 2: Type Conversion

Aim:

To demonstrate implicit and explicit type conversion.

Program:

```
#include <iostream>
using namespace std;

int main() {
```

```

int a = 10;
float b = a;

cout << "Implicit Conversion: " << b << endl;

float x = 9.8;
int y = (int)x;

cout << "Explicit Conversion: " << y;

return 0;
}

```

Conclusion:

The concept of type conversion was understood successfully.

Experiment 3: Operator Demonstration

Aim:

To demonstrate different types of operators in C++.

Program:

```

#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5;

    cout << a + b << endl;
    cout << a - b << endl;
    cout << (a > b) << endl;
    cout << ((a > b) && (b > 0)) << endl;
    cout << (a & b) << endl;

    return 0;
}

```

Conclusion:

Different operators in C++ were demonstrated successfully.

3. Control Flow Statements

Experiment 1: Grade Calculator

Aim:

To calculate grade using if-else statements.

Program:

```
#include <iostream>
using namespace std;

int main() {
    int marks;

    cout << "Enter marks: ";
    cin >> marks;

    if(marks >= 90)
        cout << "Grade A";
    else if(marks >= 75)
        cout << "Grade B";
    else if(marks >= 50)
        cout << "Grade C";
    else
        cout << "Fail";

    return 0;
}
```

Conclusion:

The program successfully calculated grades using conditional statements.

Experiment 2: Number Guessing Game

Aim:

To understand loops and conditional logic.

Program:

```
#include <iostream>
using namespace std;

int main() {
    int secret = 50;
    int guess;
```

```

while(true) {
    cout << "Guess the number: ";
    cin >> guess;

    if(guess > secret)
        cout << "Too High\n";
    else if(guess < secret)
        cout << "Too Low\n";
    else {
        cout << "Correct Guess!";
        break;
    }
}

return 0;
}

```

Conclusion:

The number guessing game was implemented successfully.

Experiment 3: Multiplication Table

Aim:

To display multiplication table using for loop.

Program:

```

#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Enter number: ";
    cin >> n;

    for(int i = 1; i <= 10; i++) {
        cout << n << " x " << i << " = " << n * i << endl;
    }

    return 0;
}

```

Conclusion:

The multiplication table was displayed successfully.

Experiment 4: Nested Control Structures

Aim:

To print a right-angled triangle using nested loops.

Program:

```
#include <iostream>
using namespace std;

int main() {
    int rows;

    cout << "Enter rows: ";
    cin >> rows;

    for(int i = 1; i <= rows; i++) {
        for(int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }

    return 0;
}
```

Conclusion:

Nested loops were implemented successfully.

4. Functions and Scope

Experiment 1: Simple Calculator Using Functions

Aim:

To perform arithmetic operations using functions.

Program:

```
#include <iostream>
using namespace std;
```

```

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int multiply(int a, int b) {
    return a * b;
}

float divide(int a, int b) {
    return (float)a / b;
}

int main() {
    int a, b;

    cout << "Enter two numbers: ";
    cin >> a >> b;

    cout << "Addition = " << add(a, b) << endl;
    cout << "Subtraction = " << subtract(a, b) << endl;
    cout << "Multiplication = " << multiply(a, b) << endl;
    cout << "Division = " << divide(a, b) << endl;

    return 0;
}

```

Conclusion:

Functions were used successfully for arithmetic operations.

Experiment 2: Factorial Using Recursion

Aim:

To calculate factorial using recursion.

Program:

```

#include <iostream>
using namespace std;

int factorial(int n) {
    if(n == 0)
        return 1;
    else

```



```
        return n * factorial(n - 1);
    }

int main() {
    int n;

    cout << "Enter number: ";
    cin >> n;

    cout << "Factorial = " << factorial(n);

    return 0;
}
```

Conclusion:

The concept of recursion was understood successfully.

Experiment 3: Variable Scope

Aim:

To demonstrate local and global variables.

Program:

```
#include <iostream>
using namespace std;

int globalVar = 100;

void display() {
    int localVar = 50;

    cout << "Global Variable = " << globalVar << endl;
    cout << "Local Variable = " << localVar << endl;
}

int main() {
    display();

    return 0;
}
```

Conclusion:

The difference between local and global scope was understood.

5. Arrays and Strings

Experiment 1: Array Sum and Average

Aim:

To calculate sum and average of array elements.

Program:

```
#include <iostream>
using namespace std;

int main() {
    int arr[5], sum = 0;
    float average;

    for(int i = 0; i < 5; i++) {
        cin >> arr[i];
        sum += arr[i];
    }

    average = sum / 5.0;

    cout << "Sum = " << sum << endl;
    cout << "Average = " << average;

    return 0;
}
```

Conclusion:

Array manipulation was performed successfully.

Experiment 2: Matrix Addition

Aim:

To perform matrix addition using 2D arrays.

Program:

```
#include <iostream>
using namespace std;

int main() {
```

```

int a[2][2], b[2][2], sum[2][2];

cout << "Enter first matrix:\n";
for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 2; j++) {
        cin >> a[i][j];
    }
}

cout << "Enter second matrix:\n";
for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 2; j++) {
        cin >> b[i][j];
    }
}

for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 2; j++) {
        sum[i][j] = a[i][j] + b[i][j];
    }
}

cout << "Sum Matrix:\n";
for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 2; j++) {
        cout << sum[i][j] << " ";
    }
    cout << endl;
}

return 0;
}

```

Conclusion:

Matrix addition using 2D arrays was implemented successfully.

Experiment 3: String Palindrome Check

Aim:

To check whether a string is palindrome or not.

Program:

```

#include <iostream>
#include <string>
using namespace std;

int main() {

```

```

    string str, rev = "";

    cout << "Enter string: ";
    cin >> str;

    for(int i = str.length() - 1; i >= 0; i--) {
        rev += str[i];
    }

    if(str == rev)
        cout << "Palindrome";
    else
        cout << "Not Palindrome";

    return 0;
}

```

Conclusion:

Palindrome checking using strings was performed successfully.

6. Introduction to Object-Oriented Programming

Experiment 1: Class for Simple Calculator

Aim:

To implement a calculator using class and objects.

Program:

```

#include <iostream>
using namespace std;

class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }

    int subtract(int a, int b) {
        return a - b;
    }

    int multiply(int a, int b) {
        return a * b;
    }
}

```

```

    }

    float divide(int a, int b) {
        return (float)a / b;
    }
};

int main() {
    Calculator c;

    cout << "Addition = " << c.add(10, 5) << endl;
    cout << "Subtraction = " << c.subtract(10, 5) << endl;
    cout << "Multiplication = " << c.multiply(10, 5) << endl;
    cout << "Division = " << c.divide(10, 5) << endl;

    return 0;
}

```

Conclusion:

Classes and objects were implemented successfully.

Experiment 2: Class for Bank Account

Aim:

To understand encapsulation in C++.

Program:

```

#include <iostream>
using namespace std;

class BankAccount {
private:
    float balance;

public:
    void setBalance(float b) {
        balance = b;
    }

    void deposit(float amount) {
        balance += amount;
    }

    void withdraw(float amount) {
        balance -= amount;
    }
}

```

```

        void display() {
            cout << "Balance = " << balance << endl;
        }
    };

    int main() {
        BankAccount b;

        b.setBalance(1000);
        b.deposit(500);
        b.withdraw(200);
        b.display();

        return 0;
    }

```

Conclusion:

Encapsulation using private data members was understood successfully.

Experiment 3: Inheritance Example

Aim:

To demonstrate inheritance in C++.

Program:

```

#include <iostream>
using namespace std;

class Person {
public:
    void displayPerson() {
        cout << "This is a Person" << endl;
    }
};

class Student : public Person {
public:
    void displayStudent() {
        cout << "This is a Student" << endl;
    }
};

class Teacher : public Person {
public:
    void displayTeacher() {
        cout << "This is a Teacher" << endl;
    }
};

```

```
};  
  
int main() {  
    Student s;  
    Teacher t;  
  
    s.displayPerson();  
    s.displayStudent();  
  
    t.displayPerson();  
    t.displayTeacher();  
  
    return 0;  
}
```

Conclusion:

Inheritance and code reusability were demonstrated successfully.